

Design by Contract

Resenha do artigo: Meyer, B. — *Design by Contract*, Cap. 1 in *Advances in Object-Oriented Software Engineering*, Prentice Hall, 1991.

Luiz Arthur Costa E Costa

Contexto e Motivação

O artigo de Bertrand Meyer, publicado como capítulo de abertura de uma coletânea sobre engenharia de software orientada a objetos, parte de uma constatação ainda muito atual: a vasta produção teórica sobre verificação formal raramente chegou à prática cotidiana do desenvolvimento de software. Pior ainda, boa parte da literatura sobre orientação a objetos simplesmente ignora as questões de corretude e robustez — como se a flexibilidade estrutural do paradigma fosse suficiente por si só. Meyer propõe então uma abordagem sistemática e pragmática para construção de software confiável, baseada na metáfora do **contrato**.

A Metáfora do Contrato

A ideia central do artigo é tratar a relação entre uma rotina (fornecedora) e quem a chama (cliente) como um contrato formal com obrigações e benefícios mútuos. Assim como em contratos civis, cada parte tem deveres claramente delimitados: o cliente deve garantir a pré-condição antes de invocar a rotina; em troca, o fornecedor compromete-se a entregar o resultado descrito na pós-condição. Se o cliente não cumpre sua parte, o fornecedor não tem obrigação alguma — e vice-versa.

Na linguagem Eiffel, esses contratos são expressos por meio de **asserções**: *preconditions* (require), *postconditions* (ensure) e *invariantes de classe* (invariant). Essas cláusulas não apenas documentam o componente, mas podem ser monitoradas em tempo de execução durante o desenvolvimento, tornando-se um poderoso mecanismo de depuração baseado em semântica de alto nível — muito superior ao teste às cegas.

Contra a Programação Defensiva

Um dos pontos mais provocativos do texto é a crítica direta à **programação defensiva** — a prática de proteger cada módulo com o máximo de verificações possível, mesmo redundantes. Para Meyer, essa abordagem aumenta a complexidade sem aumentar a confiabilidade, pois produz sistemas abarrotados de código de tratamento de erros em vez de lógica útil. A alternativa proposta é simples: definir com precisão quem é responsável por cada condição de consistência — cliente ou fornecedor — e garantir essa responsabilidade uma única vez, explicitamente, através do contrato.

"Em muitos programas existentes, mal se encontram as ilhas de processamento útil em oceanos de código de verificação de erros." — Bertrand Meyer

Uma Nova Visão sobre Exceções

A segunda grande contribuição do artigo é uma teoria coerente de tratamento de exceções, derivada diretamente da noção de contrato. Meyer critica os mecanismos de Ada e CLU por permitirem que uma rotina *aparente* ter retornado normalmente mesmo após falhar — violando o que ele chama de **Primeira Lei do Tratamento de Exceções**: uma chamada de rotina só pode terminar de duas formas, cumprindo ou não o contrato.

No mecanismo Eiffel, baseado nas cláusulas **rescue** e **retry**, uma rotina que detecta falha tem apenas dois caminhos legítimos: (1) *pânico organizado* — restaurar o invariante de classe e propagar a falha ao chamador; ou (2) *retomada* — corrigir as condições que causaram a falha e tentar novamente com a mesma ou outra estratégia. Ocultar a falha do chamador é fisicamente impossível nesse modelo, eliminando a principal fonte de comportamento inseguro observada nos exemplos Ada analisados.

Herança como Sub-Contratação

A terceira aplicação da teoria dos contratos é a reinterpretação da herança com redefinição como **sub-contratação honesta**. Ao redefinir uma rotina em uma subclasse, o sub-contratante não pode impor condições mais rígidas ao cliente (pré-condição deve ser igual ou mais fraca) nem entregar menos do que o prometido (pós-condição deve ser igual ou mais forte). Essa **Regra de Redefinição** é formalmente garantida pela sintaxe *require else / ensure then* do Eiffel, que combina automaticamente as asserções originais e redefinidas com operadores lógicos OR e AND respectivamente. A análise também evidencia por que a ligação estática (*static binding*) é incorreta por princípio: ela pode aplicar uma implementação que não preserva o invariante do objeto real em memória.

Avaliação Crítica e Relevância

O artigo apresenta uma visão coerente e bem fundamentada, com exemplos concretos em Eiffel e Ada que ilustram tanto os problemas das abordagens convencionais quanto as vantagens da abordagem contratual. A crítica à programação defensiva é especialmente valiosa e permanece atual — embora em contextos de sistemas distribuídos e concorrentes a divisão de responsabilidades entre cliente e fornecedor exija adaptações não totalmente discutidas aqui. A influência do trabalho é inegável: os conceitos de DbC foram incorporados por linguagens como D, Kotlin (contratos experimentais), Python (via bibliotecas) e frameworks de teste baseados em propriedades. A abordagem de funções parciais com pré-condições explícitas também ressoa fortemente em práticas modernas como tipagem dependente e verificação formal leve. Em suma, trata-se de uma leitura obrigatória para qualquer engenheiro de software interessado em construir sistemas corretos, robustos e bem documentados.